Sistema de Triaje Automatizado de Urgencias Médicas mediante Max-Heap y Lista Enlazada

Informe Técnico sobre Reorganización Dinámica de Prioridad y Gestión de Memoria en el Hospital General Isidro Ayora

Tercer Ciclo - Ciencias de la Computación:
Darwin Campoverde, Javier Guarnizo, Ricardo Ochoa,
Diyer Torres, Mateo Yanangomez

Carrera de Ingeniería en Ciencias de la Computación

30 de octubre de 2026

1. Título

Diseño e Implementación de un Sistema de Triaje Automatizado de Urgencias Médicas para el Hospital General Isidro Ayora Mediante una Cola de Prioridad Max-Heap y Lista Enlazada Ordenada en TypeScript/React.

2. Objetivos

2.1. Objetivo General

Evaluar el impacto algorítmico y computacional de la estructura de datos Max-Heap ($O(\log n)$) en la reorganización dinámica de la cola de espera de la sala de emergencias del Hospital General Isidro Ayora de Loja, desarrollando un prototipo web reactivo en TypeScript que reduzca la latencia en la re-priorización de pacientes ante cambios hemodinámicos y garantice la eficiencia de memoria en el motor V8 [1], [2].

2.2. Objetivos Específicos

1. **Diseñar e implementar** la estructura de datos artesanal `Max-Heap` en TypeScript con algoritmos de flotación (`heapifyUp`) y hundimiento (`heapifyDown`) para el cálculo del *Priority Score* e inversión del índice ESI (1–5) en tiempo logarítmico [1].

2. **Construir** un módulo de trazabilidad e historial clínico apoyado en una `Lista Enlazada Ordenada` ($O(1)$) que gestione el registro de pacientes atendidos y prevenga fugas de memoria (*memory leaks*) mediante la desvinculación explícita de referencias para el recolector de basura (*Garbage Collector*) del motor V8 [1], [3].

3. Problemática

El Servicio de Emergencias del Hospital General Isidro Ayora de Loja (centro de referencia de la Zona 7 de Ecuador) registra una demanda diaria sostenida de entre 110 y 140 pacientes urgentes [4]. La gestión tradicional del triaje enfrenta el dilema del reordenamiento dinámico: el estado de salud de un paciente en sala de espera no es estático; variaciones bruscas en signos vitales (SpO2, frecuencia cardíaca o presión arterial) exigen cambiar su turno de atención inmediatamente [1].

Desde la perspectiva de la teoría de algoritmos, organizar una cola de espera con arreglos desordenados implica un costo computacional de búsqueda $O(n)$ para hallar al paciente más grave [1]. Por otro lado, mantener un arreglo constantemente ordenado requiere desplazar elementos con un costo de inserción de $O(n)$ [1]. Cuando la sala alcanza una alta ocupación, estas estructuras provocan latencias impredecibles en el software, retrasando la re-categorización y la atención en situaciones donde cada minuto determina la supervivencia del usuario [1], [4].

4. Actores que Intervienen

El sistema interactúa con la estructura organizacional pública del Hospital General Isidro Ayora (Ministerio de Salud Pública del Ecuador, Zona 7) [4]:

- **Gerencia General (Dr. Andrés Benjamín Celi Cuje):** Supervisión del cumplimiento de indicadores de gestión, tiempos de espera y normativas del MSP [4].
- **Dirección Médica (Dr. Juan José Moreira):** Coordinación de la operatividad entre el triaje de admisión y las áreas de especialidad quirúrgica y cuidados intensivos [4].
- **Subdirecciones Terapéuticas (Dra. Mayra Cuenca y Bq. Verónica Morocho):** Control de soporte de laboratorio, diagnóstico y disponibilidad hospitalaria [4].
- **Responsable del Servicio de Emergencia (Dra. Karina Pindo):** Validación del protocolo ESI/Manchester y supervisión de las re-evaluaciones médicas en la sala de espera [4].
- **Personal Operativo de Salud (Médicos Evaluadores y Personal de Enfermería):** Ingreso de constantes vitales, actualización de parámetros hemodinámicos y ejecución de la llamada del paciente más grave ($O(\log n)$).

5. Requisitos Funcionales y No Funcionales

5.1. Requisitos Funcionales (RF)

- **RF-01 (Registro e Ingreso):** El sistema debe capturar la identificación del paciente, edad, nivel ESI base (1 al 5) y constantes vitales iniciales (FC, SpO2, PA, Temp, FR) [4].
- **RF-02 (Cálculo del Score):** Debe calcular un puntaje de prioridad invertido penalizando anomalías fisiológicas significativas (ej. SpO2 $< 88\,\%$, FC > 130 bpm) [1].
- **RF-03 (Reordenamiento Dinámico - Heapify):** Ante cambios en los signos vitales, el sistema debe reubicar al paciente mediante `heapifyUp` o `heapifyDown` ajustando el ranking en tiempo real [1].
- **RF-04 (Atención y Trazabilidad):** El sistema debe remover al paciente en la cima del Max-Heap ($O(\log n)$) y anexarlo al historial registrado en la Lista Enlazada Ordenada ($O(1)$) [1].

5.2. Requisitos No Funcionales (RNF)

- **RNF-01 (Eficiencia de Ejecución):** Las operaciones de re-categorización e inserción no deben exceder una complejidad de tiempo de $O(\log n)$ [1].
- **RNF-02 (Optimización de Memoria V8):** Se debe desvincular explícitamente la referencia de nodos extraídos (`nodo = null`) permitiendo la limpieza por parte del *Garbage Collector* de V8 y evitando *memory leaks* [2], [3].
- **RNF-03 (Desarrollo Frontend):** Interfaz implementada en React 18, TypeScript 5+ (modo estricto) y Tailwind CSS v4 para garantizar alta reactividad gráfica sin congelar el hilo principal (*main thread*) [2].

6. Estructuras de Datos y Algoritmos Implicados

6.1. Cola de Prioridad con Max-Heap

El **Max-Heap** se representa mediante un vector lineal (arreglo) donde cada índice i cumple con las propiedades trigonométricas de un árbol binario completo [1]:

$$\text{Padre}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{HijoIzq}(i) = 2i+1, \quad \text{HijoDer}(i) = 2i+2 \tag{1}$$

Dado que la escala ESI asigna el número 1 a la emergencia más grave y 5 a la no urgente, el software invierte la prioridad $(6 - \text{ESI})$ para posicionar al paciente de mayor gravedad en la raíz del heap (índice 0) [1].

Tabla 1: Comparativa de Complejidad Algorítmica en Selección de Pacientes [1].

Estructura	Insertar Paciente	Consultar Más Grave	Extraer Más Grave
Arreglo Desordenado	$O(1)$	$O(n)$	$O(n)$
Arreglo Ordenado	$O(n)$	$O(1)$	$O(1)$
Max-Heap (Implementado)	$\mathbf{O}(\log n)$	$\mathbf{O}(1)$	$\mathbf{O}(\log n)$

```
public insert(patient: Patient): void {
  this.heap.push(patient);
  this.heapifyUp(this.heap.length - 1);
}

private heapifyUp(index: number): void {
  while (index > 0) {
    const parentIndex = Math.floor((index - 1) / 2);
    if (this.heap[index].priorityScore <= this.heap[parentIndex].priorityScore) break;
    this.swap(index, parentIndex);
    index = parentIndex;
  }
}
```

Código 1: Algoritmo de inserción y flotación (`heapifyUp`) en TypeScript [1].

6.2. Lista Enlazada Ordenada y Memoria en el Motor V8

Para mantener el registro secuencial de pacientes atendidos sin desperdiciar bloques continuos de memoria, se utiliza una **Lista Enlazada**. Al mantener un puntero al último elemento (`tail`), el almacenamiento al final opera en tiempo constante $O(1)$ [1].

El motor JavaScript V8 subdivide la memoria en la *Call Stack* (valores primitivos y referencias de función) y el *Memory Heap* (instancias de objetos de pacientes y nodos) [2], [3]. Al atender a un paciente, se elimina la referencia en el vector Max-Heap y se migra a la lista, liberando punteros residuales para permitir que el algoritmo *Orinoco Garbage Collector* marque y limpie esa región del *Memory Heap* [3].

7. Arquitectura del Sistema

En la Figura 1 se presenta el diagrama de arquitectura por capas del sistema de triaje hospitalario (`triaje-hospital-isidro-ayora`), diseñado siguiendo un patrón modular y desacoplado para optimizar el rendimiento y la mantenibilidad del código.

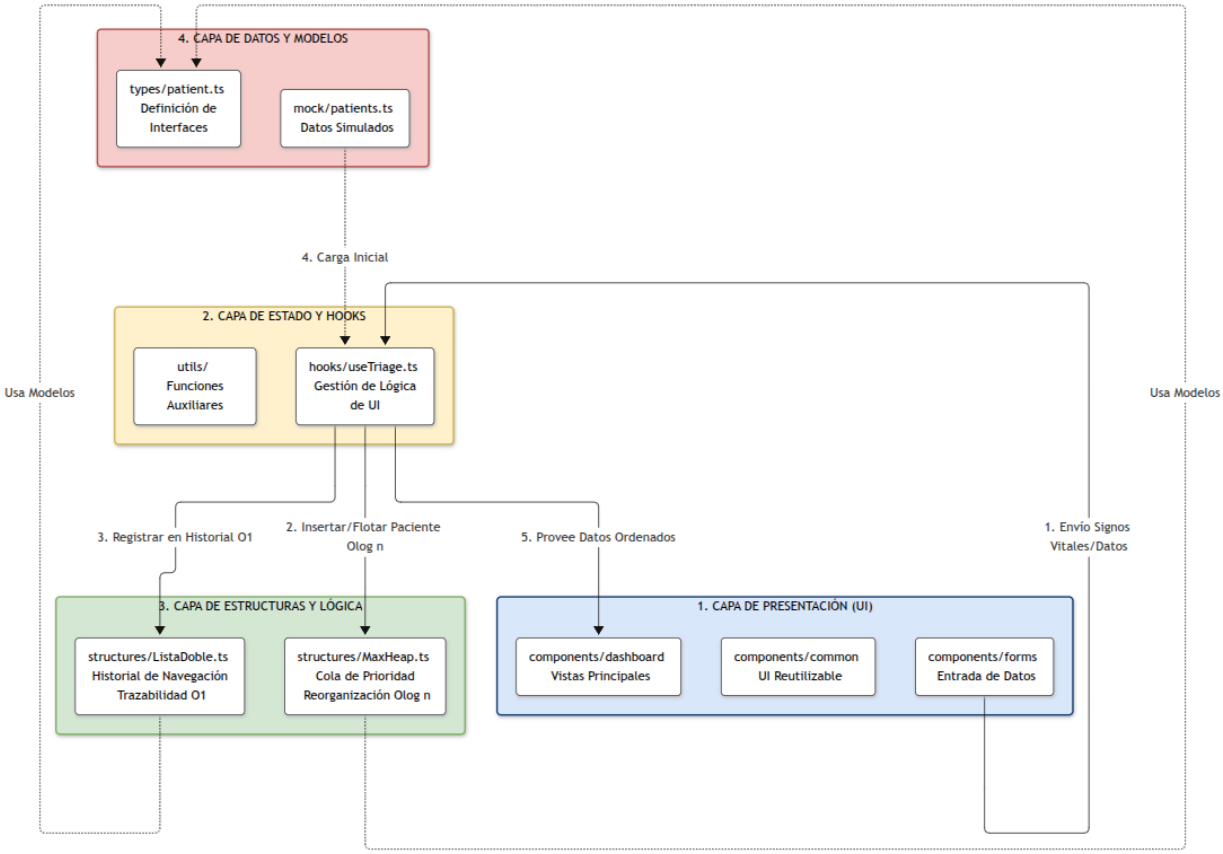


Figura 1: Arquitectura por Capas del Sistema de Triaje Hospitalario.

El sistema se estructura internamente en cuatro capas fundamentadas en las siguientes responsabilidades:

1. **Capa de Presentación (UI - React + TypeScript):** Ubicada en `src/components/`, contiene la interfaz de usuario dividida en tres submódulos: `dashboard/` para la visualización gráfica de estado, `forms/` para la captura de signos vitales del paciente, y `common/` con componentes de interfaz reutilizables.

2. **Capa de Estado y Hooks (`src/hooks/` y `src/utils/`):** Actúa como middleware de lógica de presentación. El custom hook `useTriage.ts` gestiona el ciclo de vida de los datos, responde a los eventos del usuario y sirve de puente hacia las estructuras de datos.

3. **Capa de Estructuras y Lógica (`src/structures/`):** Constituye el núcleo algorítmico del sistema. Implementa una Cola de Prioridad basada en un `MaxHeap.ts` para la reorganización eficiente de pacientes según nivel de urgencia ESI en $\mathcal{O}(\log n)$, y una `ListaDoble.ts` para la navegación e historial de pacientes en tiempo constante $\mathcal{O}(1)$.

4. **Capa de Datos y Modelos (`src/types/` y `src/mock/`):** Define los contratos de datos mediante interfaces en `patient.ts` e inicializa el sistema con datos de prueba mediante `patients.ts`.

8. Conclusiones

- La adopción del algoritmo `Max-Heap` disminuye la complejidad de reorganización de $O(n)$ a $O(\log n)$, garantizando tiempos de respuesta inmediatos ante fluctuaciones críticas de signos vitales [1].
- La combinación con una `Lista Enlazada` optimiza el ciclo de vida del objeto en el motor V8, asegurando estabilidad en el heapUsed de memoria durante largas jornadas de atención continua en el Hospital General Isidro Ayora [3], [4].

9. Bibliografía

[1] S. Guardati, *Estructuras de datos básicas: programación orientada a objetos con Java*. México D.F., México: Alpha Editorial, 2015, ISBN: 978-607-622-201-0.

[2] Mozilla Developer Network. «Memory management in JavaScript.» [En línea; accedido en febrero de 2026], MDN Web Docs. dirección: `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Memory_management`

[3] V8 Project Team. «Trash talk: the Orinoco garbage collector.» [En línea; accedido en febrero de 2026], V8 JavaScript Engine Blog. dirección: `https://v8.dev/blog/trash-talk`

[4] Ministerio de Salud Pública del Ecuador, «Informe de Gestión y Estadística del Servicio de Emergencia del Hospital General Isidro Ayora de Loja,» *Boletín Oficial de Coordinación Zonal 7*, págs. 12-18, 2026.

Entregables de Git

- **Repositorio General:** `https://github.com/TheJavier37/Repositorio_Estructura_De_Datos`
- **Repositorio del Proyecto:** `https://github.com/TheJavier37/triaje-hospital-isidro-ayora`
- **Commit Hash de Merge:** `Commit c9b0f49`

10. Evidencias de Trabajo

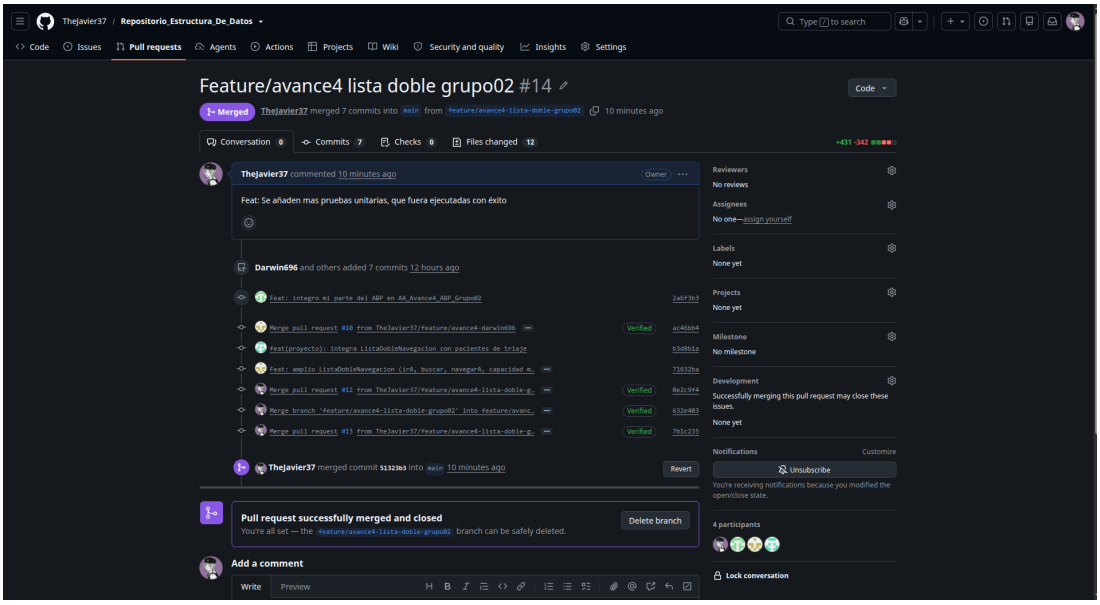


Figura 2: Evidencia del Pull Request de integración en GitHub.

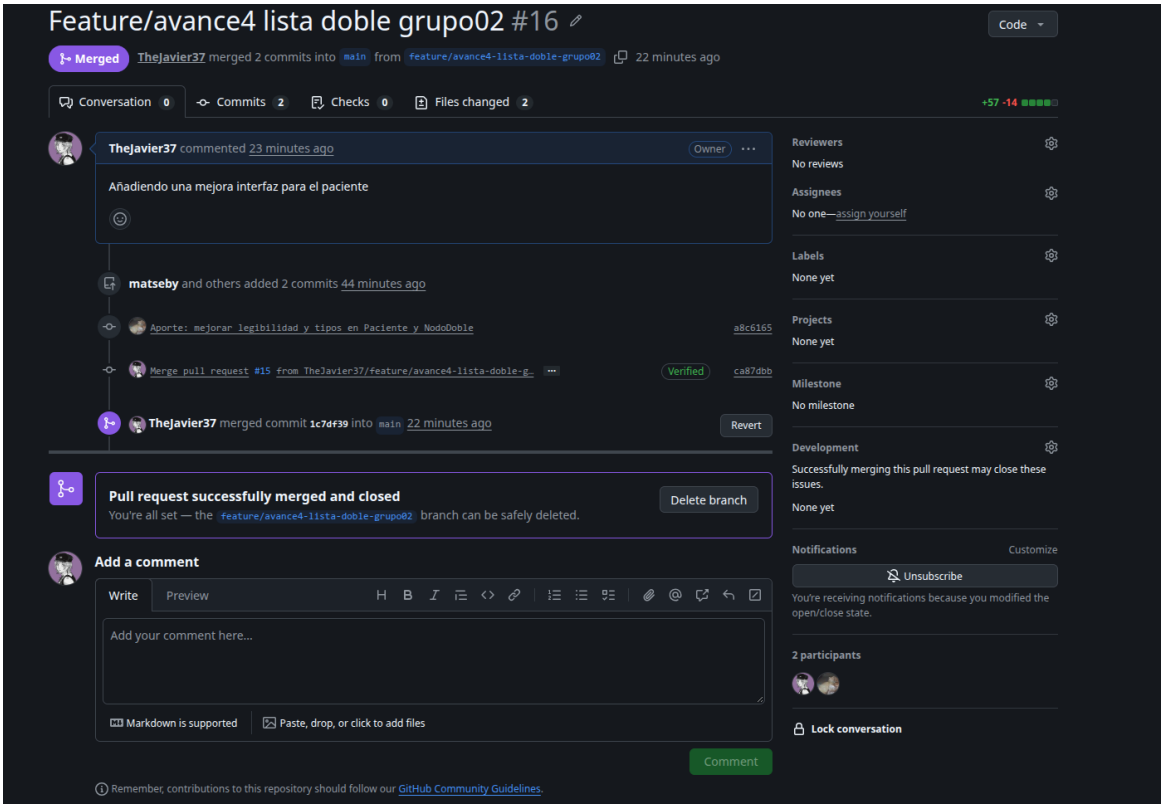


Figura 3: Integración individual del Pull Request #15 (Aporte de Mateo Yanangomez).

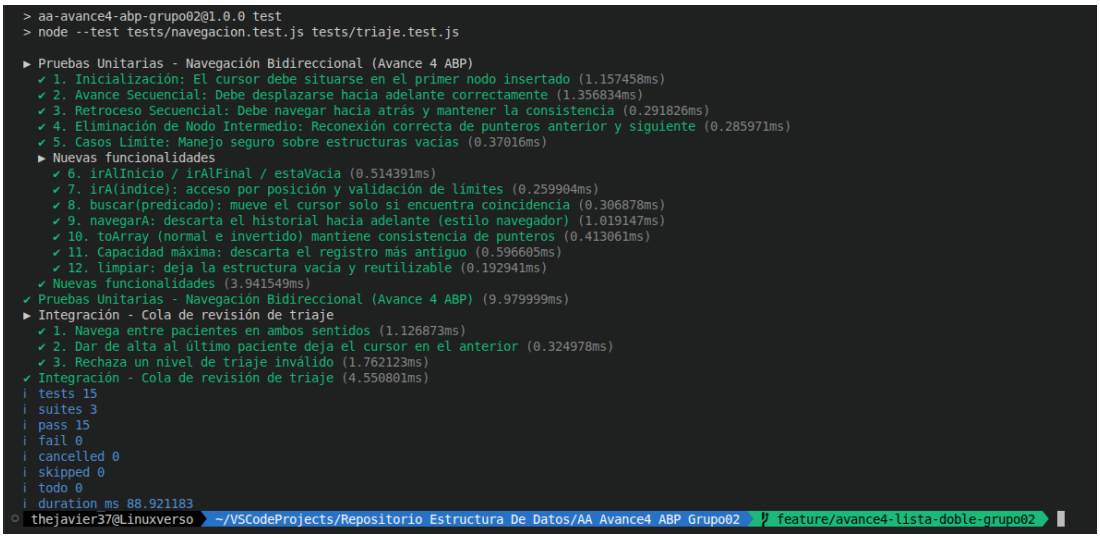


Figura 4: Ejecución exitosa de las pruebas unitarias automatizadas con `node:test`.

11. Declaración de Ética e Inteligencia Artificial

Declaramos que se hizo uso de herramientas de Inteligencia Artificial Generativa (IA) como apoyo técnico y pedagógico durante el desarrollo de este avance. La IA se empleó exclusivamente para el diseño y estructuración de las pruebas unitarias automatizadas (`tests/navegacion.test.js`), la resolución de conflictos de ramas e historiales en Git, y la optimización del formato de este informe técnico. El código de la estructura de datos principal (`ListaDobleNavegacion.js` y `NodoDoble.js`), la lógica de reconexión de punteros y la arquitectura del sistema fue analizado, diseñado, validado e integrado directamente por el equipo de desarrollo, asumiendo la autoría y responsabilidad técnica total de la solución.

Prompts Principales Utilizados

- **Pruebas Unitarias:** *"Diseña una batería de 5 pruebas unitarias en Node.js usando node:test para validar la navegación* $O(1)$ *y eliminación segura en una lista doblemente enlazada."*
- **Integración y Git:** *"¿Cómo resuelvo un conflicto de rebase en la rama feature sin perder commits antes de hacer el Pull Request?"*